

1. Signal.c

This program handles the **SIGINT** signal and displays a message when the signal is received.

It keeps running continuously until interrupted.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 signal.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <signal.h>  
  
void myhandler(int signo)  
{  
    printf("\nSignal Received\n");  
    printf("Signal Number = %d\n", signo);  
    exit(1);  
}  
  
int main()  
{  
    signal(SIGSEGV, myhandler);  
  
    int x = 0;  
  
    printf("Enter the number: ");  
    scanf("%d", &x);  
  
    printf("The entered number is %d\n", x);  
  
    return 0;  
}
```

```
sri_dhruthi@Dhruthi: ~  
sri_dhruthi@Dhruthi:~$ nano signal.c  
sri_dhruthi@Dhruthi:~$ gcc signal.c -o signal  
sri_dhruthi@Dhruthi:~$ ./signal  
Enter the number: 10  
The entered number is 10  
sri_dhruthi@Dhruthi:~$ |
```

1. Sigterm

What is the program?

This program handles the **SIGTERM** signal and displays a message when the process receives it. It also displays the process ID so you can send the signal from another terminal.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 sigterm.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
  
void handler(int sig)  
{  
    if (sig == SIGTERM)  
        printf("\nSIGTERM received!\n");  
}  
  
int main()  
{  
    signal(SIGTERM, handler);  
  
    printf("Process ID: %d\n", getpid());  
    printf("Waiting for SIGTERM...\n");  
  
    while (1)  
    {  
        sleep(2);  
    }  
  
    return 0;  
}
```

```
sri_dhruthi@Dhruthi:~$ nano sigterm.c  
sri_dhruthi@Dhruthi:~$ gcc sigterm.c -o sigterm  
sri_dhruthi@Dhruthi:~$ ./sigterm  
Process ID: 652  
Waiting for SIGTERM...  
  
SIGTERM received!  
|
```

Sigchild

What is the program?

This program handles the **SIGCHLD** signal generated when a child process terminates. It demonstrates communication between the parent and child process using signal handling.

```
sri_dhruthi@Dhruthi: ~
GNU nano 8.7.1 sigchild.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void child_handler(int sig)
{
    printf("SIGCHLD received: Child has terminated\n");
    wait(NULL);
}

int main()
{
    signal(SIGCHLD, child_handler);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child is running...\n");
        sleep(2);
        printf("Child is exiting...\n");
    }
    else
    {
        printf("Parent is waiting...\n");
        sleep(5);
        printf("Parent finished.\n");
    }

    return 0;
}
```

```
[1]+  Stopped                  ./sigterm
sri_dhruthi@Dhruthi:~$ nano sigchild.c
sri_dhruthi@Dhruthi:~$ gcc sigchild.c -o sigchild
sri_dhruthi@Dhruthi:~$ ./sigchild
Parent is waiting...
Child is running...
Child is exiting...
SIGCHLD received: Child has terminated
Parent finished.
sri_dhruthi@Dhruthi:~$ |
```

Sigpipe

This program demonstrates the **SIGPIPE** signal when a process writes to a pipe whose reading end is closed.

It shows how the signal handler responds to the closed pipe.

```
GNU nano 8.7.1 sigpipe.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigpipe(int sig)
{
    printf("\nSIGPIPE received!\n");
    printf("The reading end of the pipe is closed.\n");
    exit(1);
}

int main()
{
    int fd[2];

    signal(SIGPIPE, handle_sigpipe);

    pipe(fd);

    printf("Pipe created.\n");

    close(fd[0]);

    printf("Reading end closed.\n");

    printf("Trying to write to the pipe...\n");

    write(fd[1], "Hello", 5);

    printf("Write completed.\n");

    return 0;
}
```

```
Parent finished.
sri_dhruthi@Dhruthi:~$ nano sigpipe.c
sri_dhruthi@Dhruthi:~$ gcc sigpipe.c -o sigpipe
sri_dhruthi@Dhruthi:~$ ./sigpipe
Pipe created.
Reading end closed.
Trying to write to the pipe...

SIGPIPE received!
The reading end of the pipe is closed.
sri_dhruthi@Dhruthi:~$ |
```

Sigsegv

This program demonstrates **SIGSEGV**, which occurs when the program tries to access invalid memory.

A signal handler catches the error and displays an appropriate message.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 sigsegv.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <signal.h>  
#include <stdlib.h>  
  
void handle_sigsegv(int sig)  
{  
    printf("\nSIGSEGV received!\n");  
    printf("Invalid memory access detected.\n");  
    exit(1);  
}  
  
int main()  
{  
    signal(SIGSEGV, handle_sigsegv);  
  
    printf("Program started.\n");  
  
    int *p = NULL;  
  
    printf("Trying to access invalid memory...\n");  
  
    *p = 10;  
  
    printf("Program completed.\n");  
  
    return 0;  
}
```

```
The reading end of the pipe is closed.  
sri_dhruthi@Dhruthi:~$ nano sigsegv.c  
sri_dhruthi@Dhruthi:~$ gcc sigsegv.c -o sigsegv  
sri_dhruthi@Dhruthi:~$ ^C  
sri_dhruthi@Dhruthi:~$ ./sigsegv  
Program started.  
Trying to access invalid memory...  
  
SIGSEGV received!  
Invalid memory access detected.  
sri_dhruthi@Dhruthi:~$ |
```

